

Lab 4. Developing stored subprograms and triggers

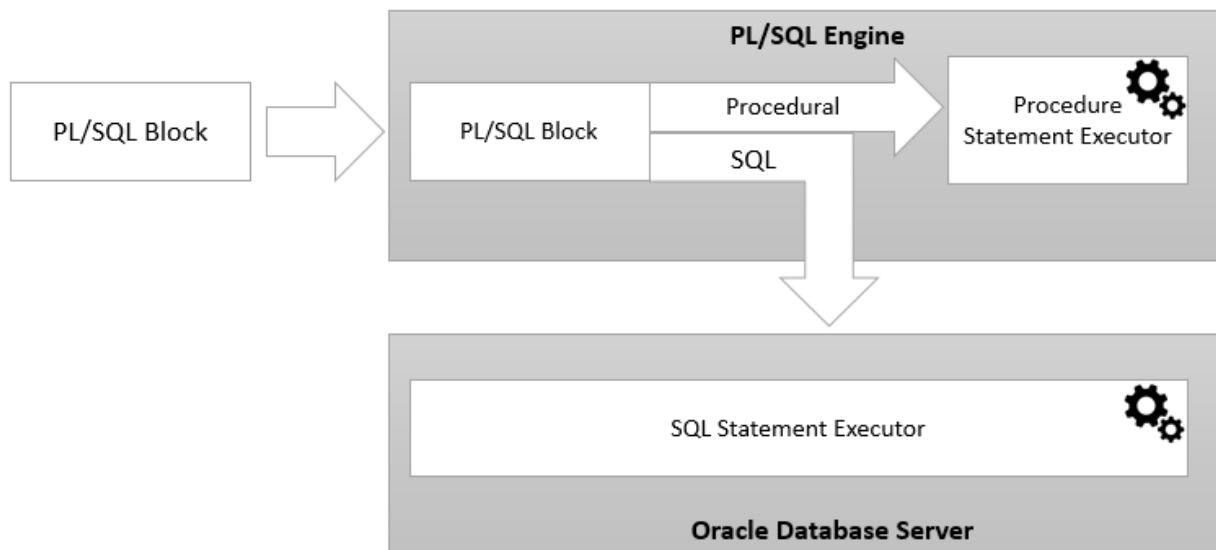
Content

- [PL/SQL Introduction](#)
- [PL/SQL anonymous block](#)
- [PL/SQL datatypes and variables](#)
- [PL/SQL conditional control](#)
- [PL/SQL loops](#)
- [Stored subprograms](#)
- [PL/SQL triggers](#)
- [Exercises](#)

PL/SQL Introduction

PL/SQL stands for “Procedural Language extensions to the Structured Query Language”. PL/SQL adds many procedural constructs to SQL language to overcome some SQL limitations. In addition, PL/SQL provides a more comprehensive programming language solution for building mission-critical applications on Oracle Databases.

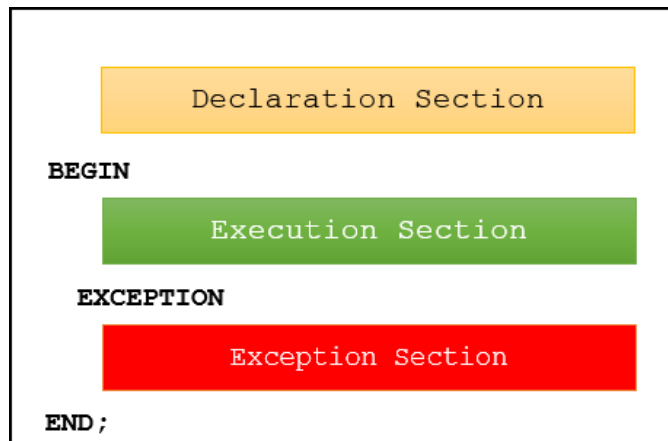
PL/SQL architecture is described in the following image:



Once we submit a PL/SQL block to the Oracle Database server, the PL/SQL engine collaborates with the SQL engine to compile and execute the code. PL/SQL engine runs the procedural elements while the SQL engine processes the SQL statements.

PL/SQL anonymous block

PL/SQL is a block-structured language whose code is organized into blocks. A block consists of three sections, as illustrated in the following figure:



- 1) Declaration section: A PL/SQL block has a declaration section where we declare variables, allocate memory for cursors, and define data types.
- 2) Executable section: A PL/SQL block has an executable section. An executable section starts with the keyword **BEGIN** and ends with the keyword **END**. The executable section must have at least one executable statement, even if it is a **NULL** statement that does nothing.
- 3) Exception-handling section: A PL/SQL block has an exception-handling section that starts with the keyword **EXCEPTION**. The exception-handling section is where we catch and handle exceptions raised by the code in the execution section.

The executable section is mandatory in a block, while the declaration and exception-handling sections are optional.

A PL/SQL block has a name. Functions and procedures are examples of named blocks. A named block is stored in the Oracle Database server and can be reusable later. A block without a name is an **anonymous block**. An anonymous block is **not saved** in the Oracle Database server, so it is just for one-time use. However, PL/SQL anonymous blocks can be useful for testing purposes.

The following example shows a simple PL/SQL anonymous block with one executable section.

```
BEGIN
    DBMS_OUTPUT.put_line ('Hello World!');
END;
```

The executable section calls the `DBMS_OUTPUT.PUT_LINE` procedure to display the “Hello World” message on the screen.

The following picture illustrates how to execute a PL/SQL block using SQL*Plus:

```

SQL> SET SERVEROUTPUT ON;
SQL> BEGIN
  2     DBMS_OUTPUT.PUT_LINE('Hello World');
  3 END;
  4 /
Hello World

PL/SQL procedure successfully completed.

```

- First, connect to the Oracle Database server using a username and password.
- Second, turn on the server output using the SET SERVEROUTPUT ON command so that the DBMS_OUTPUT.PUT_LINE procedure will display text on the screen.
- Third, type the code of the block and enter a forward slash (/) to instruct SQL*Plus to execute the block. Once you type the forward slash (/), SQL*Plus will execute the block and display the Hello World message on the screen, as shown in the illustrations.

In the next example, we first declare a variable `l_message` that holds the greeting message. And then, in the execution section, we use the `DBMS_OUTPUT.PUT_LINE` procedure to show the content of this variable instead of using a literal string.

```

DECLARE
    l_message VARCHAR2( 255 ) := 'Hello World!';
BEGIN
    DBMS_OUTPUT.PUT_LINE( l_message );
END;

```

The next anonymous block example adds an exception-handling section that catches the `ZERO_DIVIDE` exception raised in the executable section and displays an error message.

```

DECLARE
    v_result NUMBER;
BEGIN
    v_result := 1 / 0;
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE( SQLERRM );
END;

```

More details about exceptions and the list of Oracle exceptions are described in the following link:
https://www.techonthenet.com/oracle/exceptions/named_system.php

PL/SQL datatypes and variables

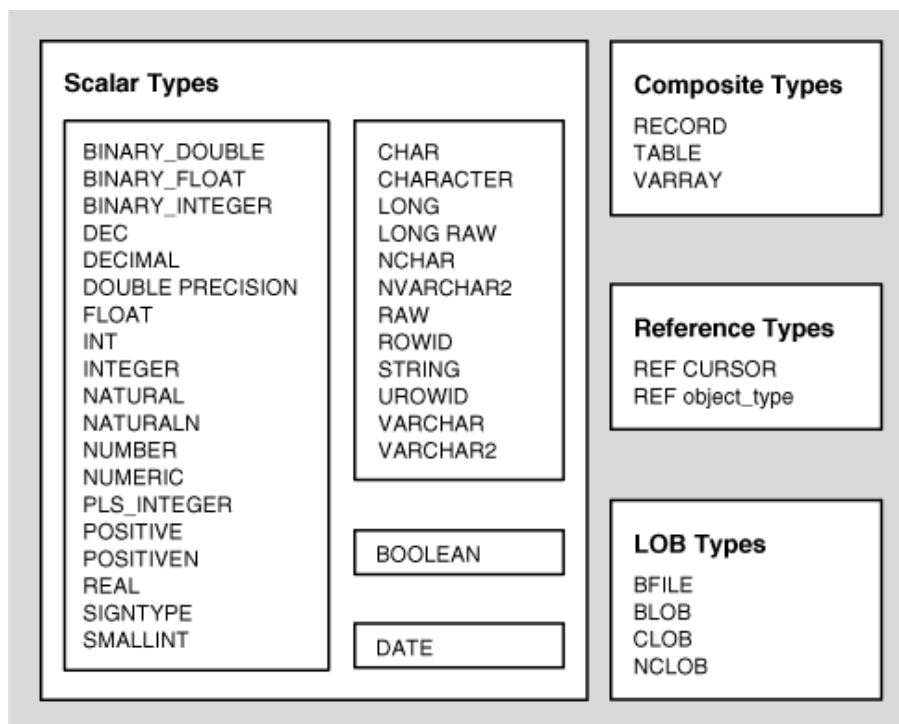
PL/SQL datatypes

Each value in PL/SQL, such as a constant, variable, and parameter, has a data type that determines the storage format, valid values and allowed operations.

PL/SQL has two kinds of data types: **scalar** and **composite**. The scalar types are types that store single values such as number, Boolean, character, and datetime, whereas the composite types are types that store multiple values, for example, record and collection.

PL/SQL divides the scalar data types into **four families**:

- Number: represents real numbers, integers, and floating-point numbers.
- Boolean: represents Boolean values (true and false). Because SQL does not have an equivalent type, we can use BOOLEAN variables and parameters in PL/SQL contexts but not inside SQL statements or queries.
- Character: lets us store alphanumeric data, represent words and text, and manipulate character strings.
- Datetime: lets us store and manipulate dates, times, and intervals (periods of time).



PL/SQL variables

In PL/SQL, a variable is named storage location, which stores a value of a particular data type. Before using a variable, we must declare it in the declaration section of a block. The syntax for a variable declaration is as follows:

```
variable_name datatype [NOT NULL] [:= initial_value];
```

The following example declares three variables `l_total_sales` and `l_contact_name`:

```
DECLARE
    l_total_sales NUMBER(15,2);
    l_contact_name VARCHAR2(255);
BEGIN
    NULL;
END;
```

PL/SQL allows us to set a default value for a variable at the declaration time. To assign a default value to a variable, we use the assignment operator (`:=`) or the `DEFAULT` keyword.

We can impose the `NOT NULL` constraint on a variable to prevent it from being assigned a `NULL` value. Besides, a variable declared with the `NOT NULL` must be initialized with a non-null value. Note that PL/SQL treats a **zero-length string** as a `NULL` value.

To declare a constant, you specify the name, `CONSTANT` keyword, data type, and the default value. The following illustrates the syntax of declaring a constant:

```
constant_name CONSTANT datatype [NOT NULL] := expression
```

For example:

```
DECLARE
    co_pi      CONSTANT REAL := 3.14159;
    co_radius  CONSTANT REAL := 10;
    co_area    CONSTANT REAL := (co_pi * co_radius**2);
BEGIN
    DBMS_OUTPUT.PUT_LINE(co_area);
END;
```

Variable assignments

To assign a value to a variable, we use the assignment operator (`:=`), for example:

```
DECLARE
    l_customer_group VARCHAR2(100) := 'Silver';
BEGIN
    l_customer_group := 'Gold';
    DBMS_OUTPUT.PUT_LINE(l_customer_group);
END;
```

Anchored declarations

Typically, we declare a variable and select a value from a table column for this variable. If the data type of the table column changes, we must adjust the program to make it work with the new type. PL/SQL allows us to declare a variable whose data type is anchored to a table column or another variable. The %TYPE attribute declares variables according to the already declared variable or database column. For example:

```
DECLARE
    l_customer_name customers.name % TYPE;
    l_credit_limit customers.credit_limit % TYPE;
BEGIN
    SELECT name, credit_limit
        INTO l_customer_name, l_credit_limit
        FROM customers
        WHERE customer_id = 38;

    DBMS_OUTPUT.PUT_LINE(l_customer_name || ':' || l_credit_limit);
END;
```

The %ROWTYPE attribute is used to declare a record type representing a row in a table. A column in a row and corresponding fields in a record have the same name and data types. For example:

```
DECLARE
    employee emp. % ROW TYPE;
BEGIN
    employee.empno := 2092;
    employee.ename := 'Sanju';
    INSERT INTO emp WHERE (empno, ename)
        VALUES (employee.empno, employee.ename);
    DBMS_OUTPUT.PUT_LINE('Row inserted');
END;
```

PL/SQL comments

PL/SQL has two comment styles: single-line and multi-line comments.

- Single-line comments: start with a double hyphen (--) that can appear anywhere on a line and extend to the end of the line.
- Multi-line comments: start with a slash-asterisk (/*) and end with an asterisk slash (*/), and can span multiple lines.

PL/SQL conditional control

The IF THEN statement

The **IF statement** allows you to either execute or skip a sequence of statements, depending on a condition. Following is the general syntax of the IF statement in Oracle:

```
IF condition_1 THEN
    statements_1
[ ELSIF condition_2 THEN
    statements_2
] ...
[ ELSE
    else_statements
]
END IF;
```

For example:

```
DECLARE
    n_sales NUMBER := 300000;
    n_commission NUMBER(10, 2) := 0;
BEGIN
    IF n_sales > 200000 THEN
        n_commission := n_sales * 0.1;
    ELSIF n_sales <= 200000 AND n_sales > 100000 THEN
        n_commission := n_sales * 0.05;
    ELSIF n_sales <= 100000 AND n_sales > 50000 THEN
        n_commission := n_sales * 0.03;
    ELSE
        n_commission := n_sales * 0.02;
    END IF;
END;
```

The CASE statement

The **CASE statement** chooses one sequence of statements to execute out of many possible sequences. There are two types of CASE statements: **simple CASE statements** and **searched CASE statements**. Both types of CASE statements support an optional ELSE clause.

The simple CASE statement has the following structure:

```
CASE selector
WHEN selector_value_1 THEN
    statements_1
WHEN selector_value_1 THEN
```

```

        statement_2
    ...
ELSE
    else_statements
END CASE;

```

Because the ELSE clause is optional, you can skip it. However, if you do so, PL/SQL will implicitly use the following:

```

ELSE
    RAISE CASE_NOT_FOUND;

```

For example:

```

CASE c_grade
WHEN 'A' THEN
    c_rank := 'Excellent' ;
WHEN 'B' THEN
    c_rank := 'Very Good' ;
...

```

The **searched CASE statement** evaluates multiple Boolean expressions and executes the sequence of statements associated with the first condition that evaluates to TRUE. The searched CASE statement has the following structure:

```

CASE
WHEN condition_1 THEN statements_1
WHEN condition_2 THEN statements_2
...
WHEN condition_n THEN statements_n
[ ELSE
    else_statements ]
END CASE;]

```

The following example illustrates using the searched CASE statement to calculate sales commission based on sales revenue.

```

CASE
WHEN n_sales > 200000 THEN
    n_commission := 0.2;
WHEN n_sales >= 100000 AND n_sales < 200000 THEN
    n_commission := 0.15;

```



```

    WHEN n_sales    >= 50000 AND n_sales < 100000 THEN
        n_commission := 0.1;
    WHEN n_sales    > 30000 THEN
        n_commission := 0.05;
    ELSE
        n_commission := 0;
    END CASE;

```

PL/SQL loops

PL/SQL supports three loop statements to repeatedly execute a block of code until a specific condition is met: the LOOP statement, the FOR loop and the WHILE loop.

LOOP statement

Here's the syntax of the PL/SQL LOOP statement:

```

<<label>> LOOP
    statements;
END LOOP loop_label;

```

The LOOP statement executes the statements in its body and returns control to the top of the loop. Typically, the body of the loop contains at least one EXIT or EXIT WHEN statement for terminating the loop. Otherwise, the loop becomes an **infinite loop**.

Typically, we use the EXIT statement with an IF statement to terminate a loop when a condition is true:

```

LOOP
    IF condition THEN
        EXIT;
    END IF;
END LOOP;

```

We can also use the EXIT WHEN statement to shorten the loop termination statement.

```

EXIT WHEN condition;

```

The following example uses the EXIT WHEN statement to terminate a loop:

```

LOOP
    l_counter := l_counter + 1;
    EXIT WHEN l_counter > 3;
    dbms_output.put_line( 'Inside loop: ' || l_counter );
END LOOP;

```

FOR LOOP statement

PL/SQL FOR LOOP executes a sequence of statements a specified number of times. The PL/SQL FOR LOOP statement has the following structure:

```
FOR index IN lower_bound .. upper_bound
LOOP
    statements;
END LOOP;
```

For example, the following loop shows a list of integers from 1 to 5.

```
BEGIN
    FOR l_counter IN 1..5
    LOOP
        DBMS_OUTPUT.PUT_LINE( l_counter );
    END LOOP;
END;
```

To reference an outer variable inside the loop, we must qualify it using a block label as below:

```
<<outer>>
DECLARE
    l_counter PLS_INTEGER := 10;
BEGIN
    FOR l_counter IN 1.. 5 loop
        DBMS_OUTPUT.PUT_LINE ('Local counter:' || l_counter);
        outer.l_counter := l_counter;
    end loop;
    -- after the loop
    DBMS_OUTPUT.PUT_LINE ('Global counter' || l_counter);
END outer;
```

Otherwise, the l_counter variable in the FOR loop hides the variable l_counter declared in the enclosing block.

To decrease the FOR loop index, we use the REVERSE keyword as follows:

```
BEGIN
    FOR l_counter IN REVERSE 1..3
    LOOP
        DBMS_OUTPUT.PUT_LINE( l_counter );
    END LOOP;
END;
```

WHILE loop statement

PL/SQL WHILE loop is a control structure that repeatedly executes a code block if a specific condition remains **true** (or **not null**).

Here's the syntax for the WHILE loop statement:

```
WHILE condition
LOOP
    statements;
END LOOP;
```

For example, the following WHILE loop executes 5 times:

```
DECLARE
    n_counter NUMBER := 1;
BEGIN
    WHILE n_counter <= 5
    LOOP
        DBMS_OUTPUT.PUT_LINE( 'Counter : ' || n_counter );
        n_counter := n_counter + 1;
    END LOOP;
END;
```

CONTINUE statement

The CONTINUE statement allows you to exit the current loop iteration and immediately continue on to the next iteration of that loop.

The following is a simple example of using the CONTINUE statement to skip over loop body execution for odd numbers:

```
BEGIN
    FOR n_index IN 1 .. 10
    LOOP
        -- skip odd numbers
        IF MOD(n_index, 2) = 1 THEN
            CONTINUE;
        END IF;
        DBMS_OUTPUT.PUT_LINE( n_index );
    END LOOP;
END;
```

Or we can use the CONTINUE WHEN statement as follows:

```

BEGIN
  FOR n_index IN 1 .. 10
  LOOP
    -- skip odd numbers
    CONTINUE
    WHEN MOD(n_index, 2) = 1 THEN

    DBMS_OUTPUT.PUT_LINE( n_index );
  END LOOP;
END;

```

PL/SQL cursor

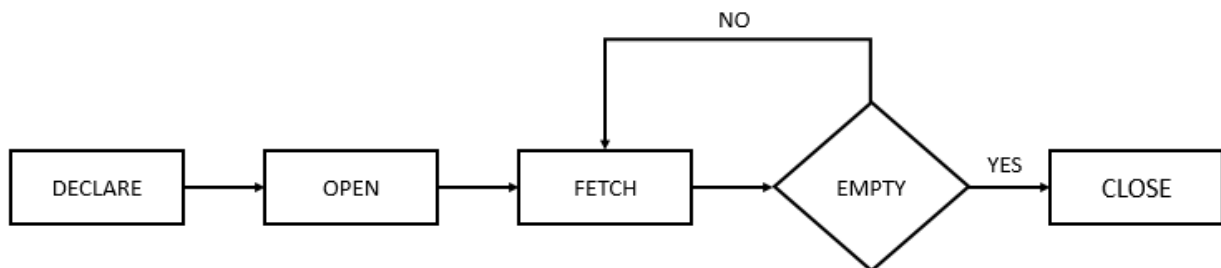
A cursor is a pointer that points to a result of a query. PL/SQL has two types of cursors: implicit cursors and explicit cursors.

Implicit cursor

Whenever Oracle executes an SQL statement such as SELECT INTO, INSERT, UPDATE, and DELETE, it automatically creates an implicit cursor. Oracle internally manages the whole execution cycle of implicit cursors and reveals only the cursor's information and statuses such as SQL%ROWCOUNT, SQL%ISOPEN, SQL%FOUND, and SQL%NOTFOUND. The implicit cursor is not elegant when the query returns zero or multiple rows which cause NO_DATA_FOUND or TOO_MANY_ROWS exception respectively.

Explicit cursor

An explicit cursor is a SELECT statement declared explicitly in the declaration section of the current block or a package specification. For an explicit cursor, we have control over its execution cycle from OPEN, FETCH, and CLOSE. Oracle defines an execution cycle that executes an SQL statement and associates a cursor with it. The execution cycle of an explicit cursor is illustrated in the following figure:



- **Declare a cursor:**

Before using an explicit cursor, we must declare it in the declaration section of a block or package as follows:

```
CURSOR cursor_name IS query;
```

- **Open a cursor:**

Before start fetching rows from the cursor, you must open it. To open a cursor, you use the following syntax:

```
OPEN cursor_name;
```

When we open a cursor, Oracle parses the query, binds variables, and executes the associated SQL statement. Oracle also determines an execution plan, associates host variables and cursor parameters with the placeholders in the SQL statement, determines the result set, and sets the cursor to the first row in the result set.

- **Fetch from a cursor:**

The FETCH statement places the contents of the current row into variables. The syntax of FETCH statement is as follows:

```
FETCH cursor_name INTO variable_list;
```

To retrieve all rows in a result set, we need to fetch each row till the last one.

- **Closing a cursor:**

After fetching all rows, you need to close the cursor with the CLOSE statement:

```
CLOSE cursor_name;
```

If you declare a cursor in an anonymous block, procedure, or function, the cursor will automatically be closed when the execution of these objects ends.

Explicit Cursor Attributes

A cursor has four attributes, which you can reference in the following format:

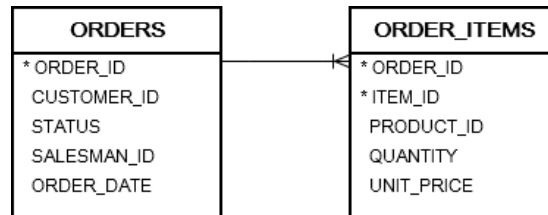
```
cursor_name%attribute
```

Attribute	Description
%ISOPEN	This attribute is TRUE if the cursor is open or FALSE if it is not.
%FOUND	This attribute has four values: ● NULL before the first fetch ● TRUE if a record was fetched successfully ● FALSE if no row is returned ● INVALID_CURSOR if the cursor is not opened
%NOTFOUND	This attribute has four values: ● NULL before the first fetch ● FALSE if a record was fetched successfully ● TRUE if no row is returned ● INVALID_CURSOR if the cursor is not opened

%ROWCOUNT	The %ROWCOUNT attribute returns the number of rows fetched from the cursor. If the cursor is not opened, this attribute returns INVALID_CURSOR.
-----------	---

PL/SQL cursor example

Given the following tables and a view that returns the sales revenues by customers:



```

CREATE VIEW sales AS
SELECT customer_id,
       SUM(unit_price * quantity) total,
       ROUND(SUM(unit_price * quantity) * 0.05) credit
FROM order_items
INNER JOIN orders USING (order_id)
WHERE status = 'Shipped'
GROUP BY customer_id;

```

Use the cursor to reset the credit limits of all customers to zero and fetch customers sorted by sales in descending order and give them new credit limits from a budget of 1 million.

The following anonymous block illustrates the logic:

```

DECLARE
    l_budget NUMBER := 1000000;

    -- declare a cursor
    CURSOR c_sales IS
        SELECT * FROM sales
        ORDER BY total DESC;

    -- declare a record
    r_sales c_sales%ROWTYPE;
BEGIN
    -- reset credit limit of all customers
    UPDATE customers SET credit_limit = 0;

    OPEN c_sales;

```

```

LOOP
    FETCH c_sales INTO r_sales;
    EXIT WHEN c_sales%NOTFOUND;

    -- update credit for the current customer
    UPDATE
        customers
    SET
        credit_limit =
            CASE
                WHEN l_budget > r_sales.credit THEN
                    r_sales.credit
                ELSE l_budget
            END
    WHERE
        customer_id = r_sales.customer_id;

    -- reduce the budget for credit limit
    l_budget := l_budget - r_sales.credit;

    DBMS_OUTPUT.PUT_LINE( 'Customer id: ' || r_sales.customer_id
        || ' Credit: ' || r_sales.credit || ' Remaining Budget: '
        || l_budget );

    -- check the budget
    EXIT WHEN l_budget <= 0;
END LOOP;

CLOSE c_sales;
END;

```

Stored subprograms

Subprograms are named PL/SQL blocks that can take parameters and be invoked. Subprograms are either **functions** or **procedures** and can be compiled and stored in an Oracle database, ready to be executed. Once compiled, it is a schema object known as a stored procedure or stored function. Generally, we use a function to compute a value and a procedure to perform a business action.

PL/SQL procedure

The following illustrates the basic syntax of **creating a procedure** in PL/SQL:

```

CREATE [OR REPLACE ] PROCEDURE procedure_name (parameter_list)
IS
    [declaration statements]

```

```

BEGIN
    [execution statements]
EXCEPTION
    [exception handler]
END [procedure_name ];

```

Each parameter can be in either IN, OUT, or INOUT mode. The parameter mode specifies whether a parameter can be read from or written to.

1. An IN parameter is read-only. We can reference an IN parameter inside a procedure but cannot change its value. Oracle uses IN as the default mode. This means that if we don't explicitly specify the mode for a parameter, Oracle will use IN mode.
2. An OUT parameter is writable. Typically, we set a returned value for the OUT parameter and return it to the calling program. Note that a procedure ignores the value that we supply for an OUT parameter.
3. An INOUT parameter is both readable and writable. The procedure can be read and modified.

Similar to an anonymous block, the procedure body has **three parts**. The executable part is mandatory, whereas the declarative and exception-handling parts are optional. The executable part must contain at least one executable statement.

1. **Declarative part:** In this part, we can declare variables, constants, cursors, etc. Unlike an anonymous block, a declaration part of a procedure does not start with the DECLARE keyword.
2. **Executable part:** This part contains one or more statements that implement specific business logic. It might contain only a NULL statement.
3. **Exception-handling part:** This part contains the code that handles exceptions.

The following procedure accepts a customer ID and prints out the customer's contact information, including first name, last name, and email:

```

CREATE OR REPLACE PROCEDURE print_contact(in_customer_id NUMBER)
IS
    r_contact contacts % ROWTYPE;
BEGIN
    -- Get contact based on customer ID
    SELECT *
        INTO r_contact
        FROM contacts
        WHERE customer_id = p_customer_id;

    -- print out contact's information
    dbms_output.put_line( r_contact.first_name || ' ' ||
        r_contact.last_name || '<' || r_contact.email || '>' );

```



```

EXCEPTION
    WHEN OTHERS THEN
        dbms_output.put_line(SQLERRM);
END;

```

The following shows the syntax for **executing a procedure**:

```
EXEC[UTE] procedure_name(arguments);
```

For example, to execute the `print_contact` procedure that prints the contact information of customer ID 100, we use the following statement:

```
EXEC print_contact(100);
```

To **delete a procedure**, we use the `DROP PROCEDURE` followed by the procedure's name that we want to drop, as shown in the following syntax:

```
DROP PROCEDURE procedure_name;
```

PL/SQL function

Similar to a procedure, a PL/SQL function is a reusable program unit stored as a schema object in the Oracle Database. However, a function will return a value to the function call. The following illustrates the syntax for **creating a function**:

```

CREATE [OR REPLACE] FUNCTION function_name (parameter_list)
    RETURN return_type
IS
    [declarative section]

BEGIN
    [executable section]

    [EXCEPTION]
        [exception-handling section]

END;

```

In which:

- The **RETURN** clause specifies the datatype of the returned value.
- Each **parameter** of the function can be either in the IN, OUT, or INOUT mode similar to the procedure declaration.

- The **declarative section**: where we declare variables, constants, cursors, and user-defined types.
- The **executable section**: where we place the executable statements. We must have at least one RETURN statement in the executable statement.
- The **exception-handling section**: where we put the exception handler code.

The following example creates a function that calculates total sales by year:

```
CREATE OR REPLACE FUNCTION get_total_sales(in_year PLS_INTEGER)
RETURN NUMBER
IS
    l_total_sales NUMBER := 0;
BEGIN
    -- get total sales
    SELECT SUM(unit_price * quantity) INTO l_total_sales
    FROM order_items
    INNER JOIN orders USING (order_id)
    WHERE status = 'Shipped'
    GROUP BY EXTRACT(YEAR FROM order_date)
    HAVING EXTRACT(YEAR FROM order_date) = in_year;

    -- return the total sales
    RETURN l_total_sales;
END;
```

We can use a function anywhere that we use an expression of the same type. We can **call a function** in various places, such as:

- Assignment statement:

```
DECLARE
    l_sales_2017 NUMBER := 0;
BEGIN
    l_sales_2017 := get_total_sales (2017);
    DBMS_OUTPUT.PUT_LINE('Sales 2017: ' || l_sales_2017);
END;
```

- Boolean expression:

```
BEGIN
    IF get_total_sales (2017) > 10000000 THEN
        DBMS_OUTPUT.PUT_LINE('Sales 2017 is above target');
    END IF;
END;
```

- SQL statement:

```
SELECT get_total_sales(2017) FROM dual;
```

We can delete a function from Oracle DB using the DROP FUNCTION statement:

```
DROP FUNCTION function_name;
```

PL/SQL triggers

What is a trigger?

A trigger is a named PL/SQL block stored in the Oracle Database and executed automatically when a triggering event takes place. The event can be any of the following:

- A data manipulation language (DML) statement executed against a table e.g., INSERT, UPDATE, or DELETE. For example, if we define a trigger that fires before an INSERT statement on the Customers table, the trigger will fire once before a new row is inserted into the Customers table.
- A data definition language (DDL) statement executes e.g., CREATE or ALTER statement. These triggers are often used for auditing purposes to record changes in the schema.
- A system event such as startup or shutdown of the Oracle Database.
- A user event such as login or logout.

We cannot explicitly invoke a trigger; however, we can enable and disable a trigger.

Oracle trigger usages

Oracle triggers are useful in many cases, such as the following:

- Enforcing complex business rules that cannot be established using integrity constraints such as UNIQUE, NOT NULL, and CHECK.
- Preventing invalid transactions.
- Gathering statistical information on table accesses.
- Generating value automatically for derived columns.
- Auditing sensitive data.

Creating a trigger

To create a new trigger in Oracle, we use the following CREATE TRIGGER statement:

```
CREATE [OR REPLACE] TRIGGER trigger_name
    {BEFORE | AFTER } triggering_event ON table_name
    [FOR EACH ROW]
    [FOLLOWS | PRECEDES another_trigger]
    [ENABLE / DISABLE ]
    [WHEN condition]
DECLARE
    declaration statements
BEGIN
    executable statements
```

```

EXCEPTION
    exception_handling statements
END;

```

The above syntax can be explained as follows:

- **BEFORE | AFTER:** The BEFORE or AFTER option specifies when the trigger fires, either before or after a triggering event e.g., INSERT, UPDATE, or DELETE.
- **ON table_name:** The table_name is the name of the table associated with the trigger.
- **FOR EACH ROW:** The clause FOR EACH ROW specifies that the trigger is a **row-level trigger**. A row-level trigger fires once for each row inserted, updated or deleted. Besides the row-level triggers, we have **statement-level triggers**. A statement-trigger fires once, regardless of the number of rows affected by the triggering event. If you omit the FOR EACH ROW clause, the CREATE TRIGGER statement will create a statement-level trigger.
- **ENABLE | DISABLE:** The ENABLE / DISABLE option specifies whether the trigger is created in the enabled or disabled state. Note that if a trigger is disabled, it is not fired when the triggering event occurs. By default, if you don't specify the clause ENABLE / DISABLE, the trigger is created with the enabled state.
- **FOLLOWS | PRECEDES another_trigger:** For each triggering event e.g., INSERT, UPDATE, or DELETE, you can define multiple triggers to fire. In this case, you need to specify the firing sequence using the FOLLOWS or PRECEDES option.

Creating a trigger example

Given a table with the following structure:

```

CREATE TABLE audits (
    audit_id          NUMBER PRIMARY KEY,
    table_name        VARCHAR2(255),
    transaction_name  VARCHAR2(10),
    by_user           VARCHAR2(30),
    transaction_date  DATE
);

```

Now, we create a new trigger associated with the Customers table that records the update and delete event:

```

CREATE OR REPLACE TRIGGER customers_audit_trg
    AFTER UPDATE OR DELETE
    ON customers
    FOR EACH ROW
DECLARE
    l_transaction VARCHAR2(10);
BEGIN
    -- determine the transaction type

```

```

l_transaction := CASE
    WHEN UPDATING THEN 'UPDATE'
    WHEN DELETING THEN 'DELETE'
END;

-- insert a row into the audit table
INSERT INTO audits (table_name, transaction_name,
                    by_user, transaction_date)
VALUES('CUSTOMERS', l_transaction, USER, SYSDATE);
END;

```

The clause `AFTER UPDATE OR DELETE ON customers` will fire the trigger after a row in the table Customers is updated or deleted.

Oracle trigger levels

Oracle supports 2 trigger levels: statement-level and row-level trigger. A **statement-level trigger** is fired whenever a trigger event occurs on a table, regardless of how many rows are affected. In other words, a statement-level trigger executes once for each transaction. By default, the statement `CREATE TRIGGER` creates a statement-level trigger when you omit the `FOR EACH ROW` clause. Note that Oracle doesn't support a mechanism to access all affected records in the trigger like some of the DBMS. Therefore, they don't inherently know which rows were affected. We have to use intermediate staging tables or additional application logic to handle multiple rows.

Row-level triggers fires once for each row affected by the triggering event, such as `INSERT`, `UPDATE`, or `DELETE`. Row-level triggers are useful for data-related activities such as data auditing and data validation. To create a new row-level trigger, you use the `CREATE TRIGGER` statement with the `FOR EACH ROW` clause. Because row-level triggers execute within the context of a single row, we can access the old and new column values using the following syntax:

```

:OLD.column_name
:NEW.column_name

```

For example, this statement checks the value of the `credit_limit` column to see if the new credit is greater than the current credit:

```

IF :NEW.credit_limit > :OLD.credit_limit THEN
    -- carry an action
END;

```

This table illustrates the availability of the `:NEW` and `:OLD` variables by the triggering event:

Trigger event	:NEW	:OLD
INSERT	Yes	No

UPDATE	Yes	Yes
DELETE	No	Yes

A row-level trigger fires each time a row is affected by a triggering event. For example, if you update 1000 rows in a table, the trigger will fire 1000 times, which potentially cause a performance issue. To specify a condition of when to fire the trigger, we can use the WHEN clause. For example, the following trigger only fires when you update credit for a customer with a new credit greater than 10,000:

```
CREATE OR REPLACE TRIGGER
  BEFORE UPDATE OF credit_limit
  ON customers
  FOR EACH ROW
  WHEN NEW.credit_limit > 10000;
...
```

The following example creates a row-level trigger that prevents users from updating credit for a customer if the new credit increases to more than double:

```
CREATE OR REPLACE TRIGGER customers_update_credit_trg
  BEFORE UPDATE OF credit_limit
  ON customers
  FOR EACH ROW
  WHEN (NEW.credit_limit > 0)
BEGIN
  -- Check the credit limit
  IF :NEW.credit_limit >= 2 * :OLD.credit_limit THEN
    raise_application_error(-20101, 'The new credit '
      || :NEW.credit_limit
      || ' cannot increase to more than double, the current
credit '
      || :OLD.credit_limit);
  END IF;
END;
```

Note that Oracle will rollback the update because we call the raise_application_error procedure inside the trigger. The raise_application_error has the following syntax:

```
raise_application_error(
  error_number,
  message
  [, {TRUE | FALSE}]
```

);

In this syntax:

- The error_number is a negative integer with the range from -20999 to -20000.
- The message is a character string that represents the error message. Its length is up to 2048 bytes.
- If the third parameter is FALSE, the error replaces all previous errors. If it is TRUE, the error is added to the stack of previous errors.

Exercises

Procedure/Function exercises

1. Write PL/SQL code to create a function to check if a number is prime.
2. Write PL/SQL code to create a procedure to calculate the factorial of a number.
3. Write a PL/SQL package that contains a function to calculate the bonus amount for employees based on their salary and years of service.

Given the structure of the Employee table is as follows:

employee_id	integer
first_name	varchar(25)
last_name	varchar(25)
email	varchar(25)
phone_number	varchar(15)
hire_date	date
job_id	varchar(25)
salary	integer
commission_pct	decimal(5,2)
manager_id	integer
department_id	integer

4. Write a PL/SQL that creates a procedure to calculate the average salary of employees in a specific department.
5. Write a PL/SQL that creates a function to retrieve the total number of employees in a specific job title.
6. Write a PL/SQL that creates a procedure to update an employee's salary based on their performance rating.

Trigger exercises

1. Write a code in PL/SQL to create a trigger that automatically updates a 'last_modified' timestamp whenever a row in a specific table is updated for the following table:

```
CREATE TABLE employee (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50),
```

```

        last_modified TIMESTAMP
    );

```

2. Write a code in PL/SQL to create a trigger that prevents updates on a certain column during specific hours of the day for the following table:

```

CREATE TABLE orders (
    order_id NUMBER PRIMARY KEY,
    order_date TIMESTAMP,
    order_amount NUMBER
);

```

3. Write a code in PL/SQL to implement a trigger that maintains a transaction history log whenever a row is deleted from the following table:

```

CREATE TABLE employees (
    employee_id NUMBER PRIMARY KEY,
    first_name VARCHAR2(50),
    last_name VARCHAR2(50)
);

```

4. Write a code in PL/SQL to develop a trigger that enforces referential integrity by preventing the deletion of a parent record if child records exist for the following table:

```

CREATE TABLE departments (
    department_id NUMBER PRIMARY KEY,
    department_name VARCHAR2(50)
);

CREATE TABLE employees (
    employee_id NUMBER PRIMARY KEY,
    first_name VARCHAR2(50),
    last_name VARCHAR2(50),
    department_id NUMBER,
    CONSTRAINT fk_dept FOREIGN KEY (department_id)
        REFERENCES departments (department_id)
);

```

5. Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found for the following table:

```

CREATE TABLE products (
    product_id NUMBER PRIMARY KEY,
    product_name VARCHAR2(50)
);

```